# Database Management (1): Analysis

## Relational Data Model

Ling-Chieh Kung

Department of Information Management
National Taiwan University

# Road Map

- **A motivating example**
- Entity-relationship modeling
- Creating a relational database schema

# Data modeling

- Imagine that you are hired by a big company, which owns an information system with a huge database.
  - Thousands of data tables, thousands of business rules, etc.

# An example database: RETAIL

- Consider a **retail** business that hires many **employees** to run many **stores**, selling a variety of **products**.
- For each store, we store its store ID, address, and phone number.
  - Each store has a unique store ID and a unique address.
  - Each store's address is composed of a postal code and an address string.
  - Each store may have several phone numbers.
  - Each store is managed by exactly one employee.
  - Each store sells several products.
  - Each store records several **sales transactions**.

# An example database: RETAIL

- For each product, we store its product ID and name.
  - Each product has a unique ID and a unique name.
  - Each product may be sold by several stores.
  - Each product's price may vary over time and across different stores.
  - Each product may be involved in several sales transactions, with the sales quantity and unit price recorded.
- For each employee, we store his/her employee ID, name, gender, birthday, and monthly salary.
  - Each employee has a unique employee ID.
  - Each employee works in exactly one store.
  - Each employee is supervised by at most one another employee.

# An example database: RETAIL

- For each sales transaction, we store its receipt number and date time.
  - Each sales transaction has a unique receipt number.
- Each sales transaction may include multiple products.
  - Each product is sold at a unit price, which may be different from transaction to transaction.
  - Each product may be sold by one or multiple units.

- As an analyst, how to **analyze the data** to generate business insights?
- As a designer or developer, how to **create a database** to record the operations of this retail business?

# Data modeling

- To **analyze** the data in the database, you need to first **understand** how data are stored in the database and the underlying business rules.
  - You need to understand the **data model** (資料模型) that describes the data.
- If you want to develop an information system or a new module based on an existing one, you do **data modeling** (資料建模) by studying the data, their relationship, and the business rules to construct an appropriate data model.
- The model used in relational databases is the **relational data model** (關聯式資料模型), which comes from the **entity-relationship model** (實體關聯模型).
  - Let's introduce both of them.

# Road Map

- A motivating example
- **Entity-relationship modeling**
  - **Entities**
  - Relationships
  - Entity-relationship diagram
- Creating a relational database schema

# Conceptual data modeling

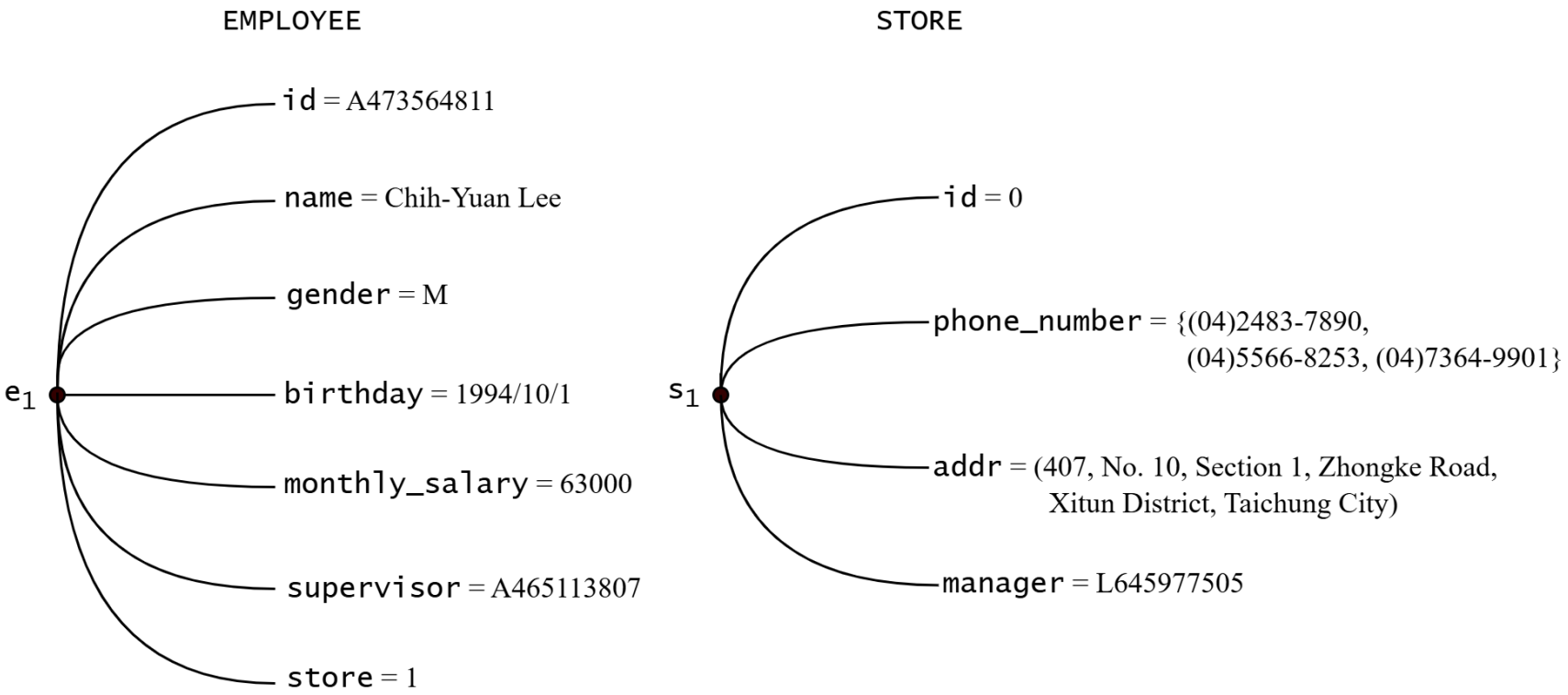
- To design a database for an application, we start from **conceptual data modeling** (概念性資料建模) using **entities** (實體) and **relationships** (關聯).
  - To create a high-level conceptual data model.
- People do **entity-relationship modeling** (**ER modeling**, 實體關聯建模).
  - This will help us create a concrete data model, which is the relational model we want.

# Entities

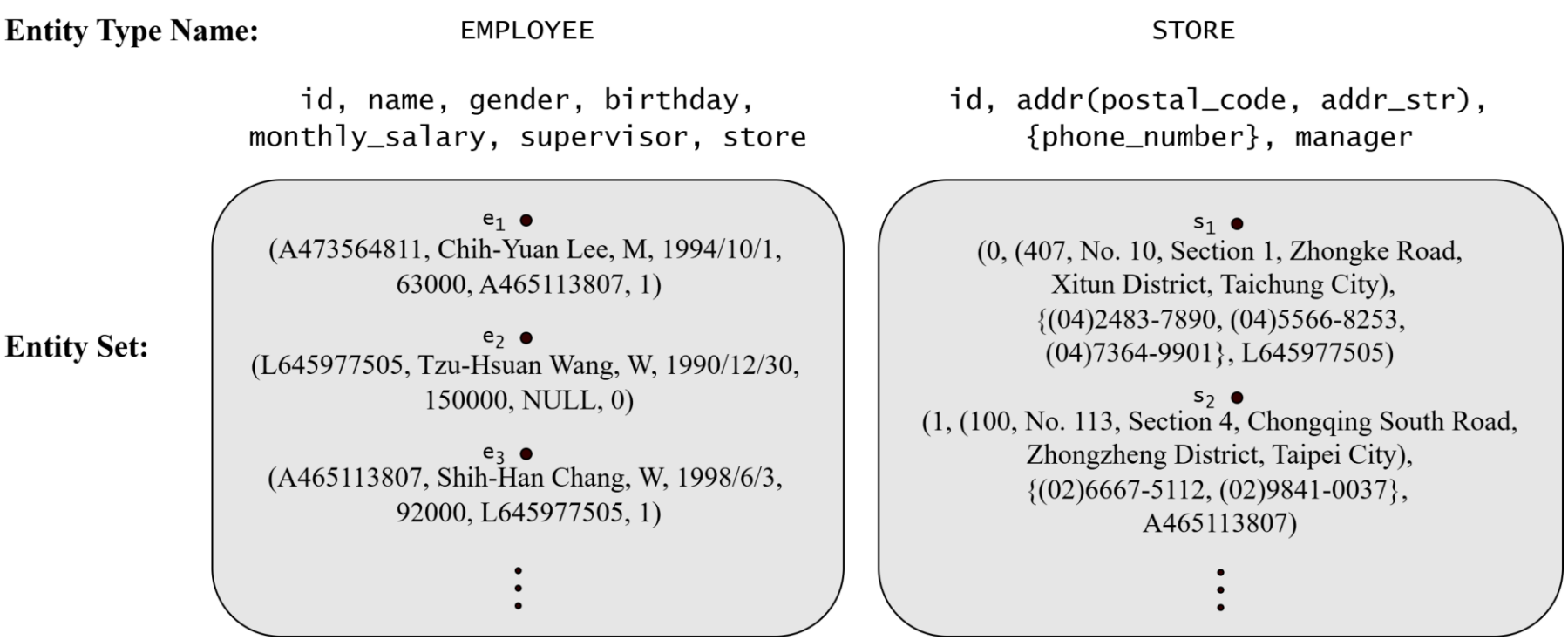
- A database records information of many **entities** (實體).
  - An entity is a "thing" in the real world (physical or conceptual).
  - For example, a physical train, a train route, a scheduled train (or a train trip), a physical member, a ticket-purchasing transaction, etc.
- To be more specific:
  - An **entity instance** (實體物件) is a specific item, entry, object, thing, or record.
  - The schema of an **entity type** (實體類別) defines a single structure and a set of **attributes** (屬性) shared by all entity instances.
- An attribute is a particular property (of an entity).
  - The same type of entity instances should have the same set of attributes.

# Two example entities $e_1$ and $s_1$

- Let `EMPLOYEE` and `STORE` be two **entity types**.
  - Two example **entity instances** are $e_1$ and $s_1$.



# Entity types and entity instances



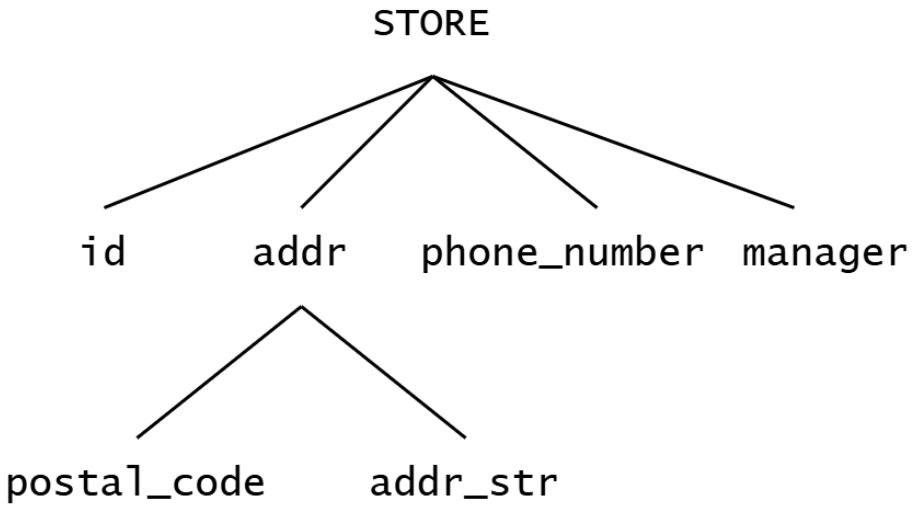
# Similarity between entities and OOP

- If we try to understand entities in the terminology of object-oriented programming (OOP):

| Entity | OOP |
|---|---|
| Entity type | Class |
| Entity instance | Object |
| Attribute | Instance variable |

# Attributes and attribute value

- There are different kinds of attributes and attribute values.
- An attribute value may be **simple** or **composite**.
  - For example, a store's ID is typically simple, but a store's address may be composite (including the postal code, address string, etc.).
  - As an example, the `STORE` address, which is `addr`, is composed of `postal_code` and `addr_str`.



# Attributes and attribute value

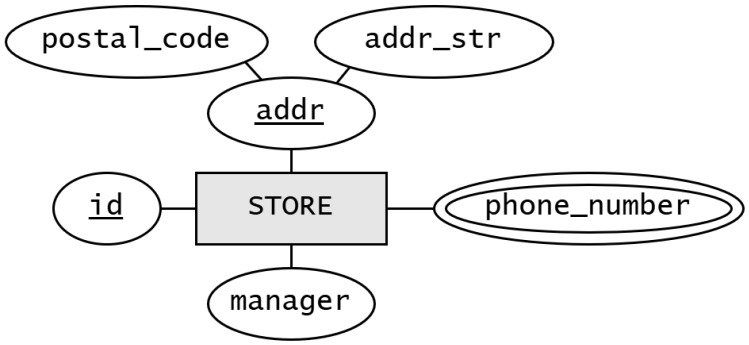
- An attribute may be **single-valued** or **multi-valued**.
  - For example, a store has only one address but may have multiple phone numbers.
- An attribute value may be **stored** or **derived**.
  - For example, an employee's birthday is stored, but her/his age is derived.
- An entity instance's value of an attribute may be **null** (空值), which means that that information is missing for that entity instance.
  - For example, there may be employees who have no supervisor.

# Keys and value sets

- An important constraint on the entity instances of an entity type is the **key** (鍵).
  - If an attribute of an entity type should satisfy the **uniqueness constraint** (唯一性限制), i.e., two entity instances **cannot have the same value** in it, this attribute is a key.
  - For example, two courses cannot have the same course number, and thus the attribute course number is a key. However, course name is not a key because two courses may have the same name.
  - Each entity instance can be **uniquely defined** by its key value, and one may always **search** for a specific entity instance using a key.
- Each simple attribute of an entity type is associated with a **value set** (or **domain** of values, 定義域).
  - For example, the value set of the "month" attribute may be {1, 2, … , 12}.

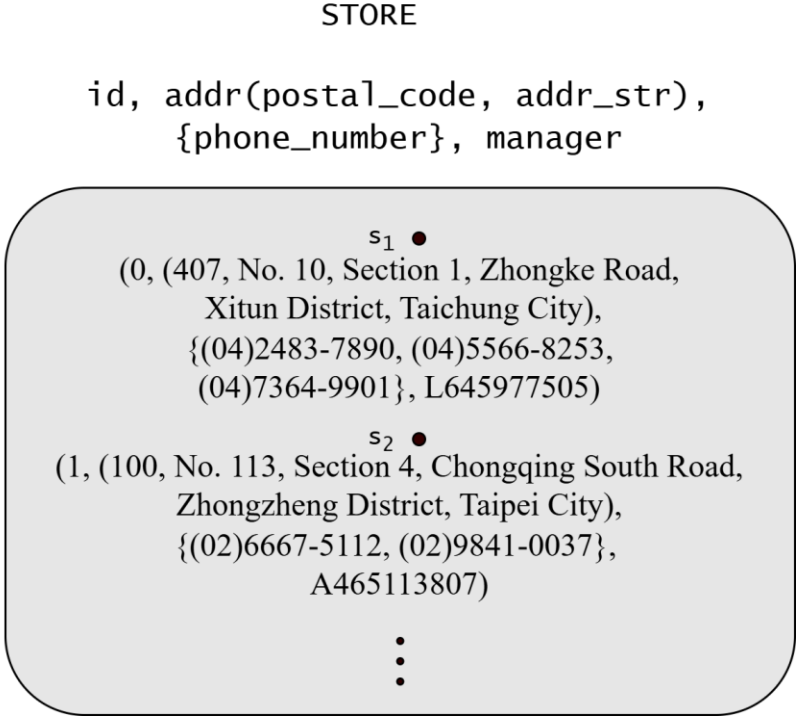
# Visualizing an entity type STORE

- An **entity type** may be **visualized** using different shapes to denote different concepts.
  - Rectangle: type.
  - Oval: attribute.
  - Double oval: multivalued attribute.
  - Underline: key.
  - Branch: composite attribute.

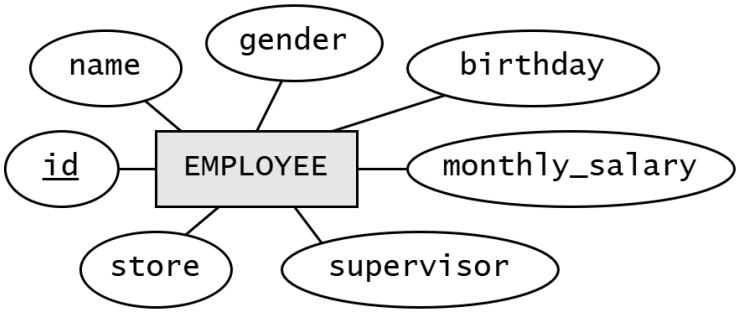


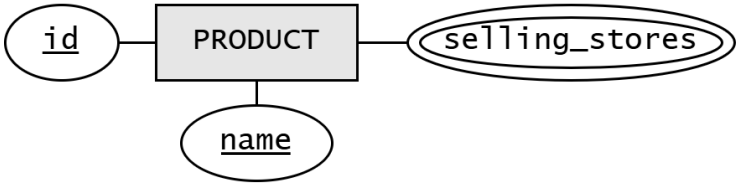
# Listing entity instances of the type STORE

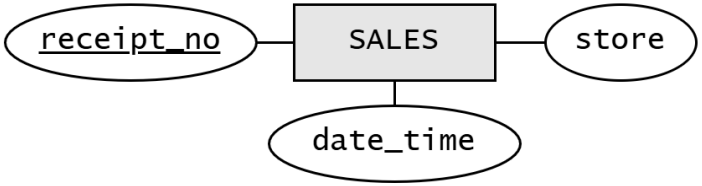
- **Entity instances** may be **listed** one by one.
- Common notations for listing entity instances:
  - Parenthesis: composite value.
  - Curly bracket: multiple value.

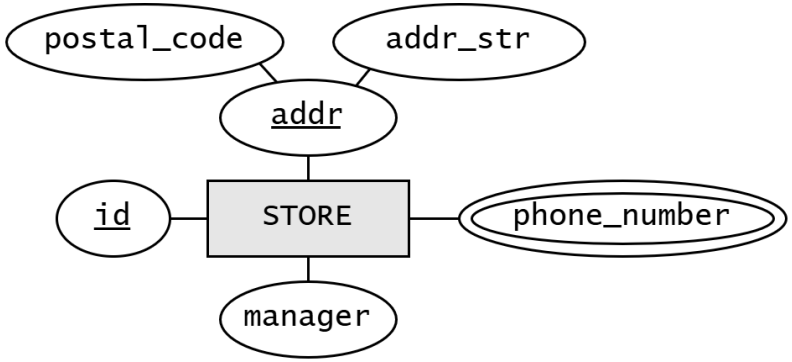


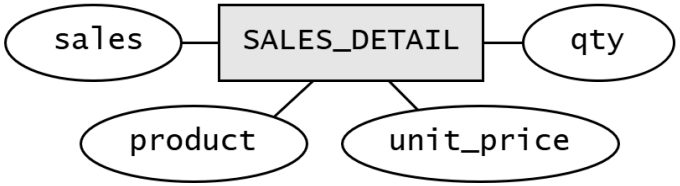
# Preliminary design of RETAIL



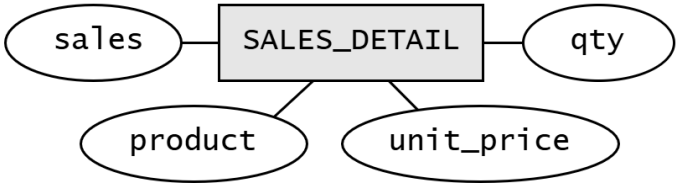



# Strong and weak entity types

- Note that the entity type `SALES_DETAIL` **has no key attributes**!
  - Each record in `SALES_DETAIL` includes the information (unit price and quantity) for a **product** in a **sales transaction**.




  - Each record in `SALES_DETAIL` cannot exist without knowing which sales transaction and which product it belongs to (is associated with).
- Entity types may be strong or weak:
  - An entity type that has its own key attributes is a **regular (strong) entity type** (強實體類別).
  - An entity type whose attributes are not enough to form its key attributes is a **weak entity type** (弱實體類別).
- We will learn how to relate weak entity types with strong ones soon.

# Road Map

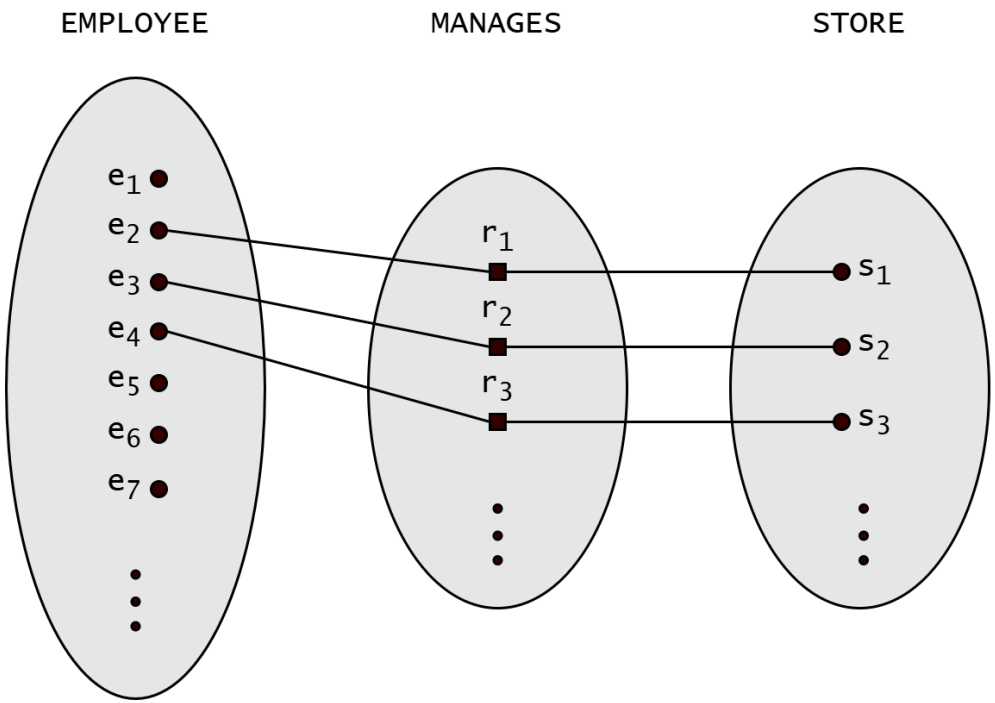
- A motivating example
- Entity-relationship modeling
  - Entities
  - **Relationships**
  - Entity-relationship diagram
- Creating a relational database schema

# Relationships

- Entities are connected by **relationships** (關聯).
- Relationships denote **actions** that entities may take on other entities.
  - For example, a student takes a course, an instructor offers a course, an instructor belongs to one or several departments, etc.
  - For example, an employee manages a store, an employee works in a store, an employee supervises another employee, etc.
- Just like the existence of entity types and entity instances, there are **relationship types** (關聯類別) and **relationship instances** (關聯物件).

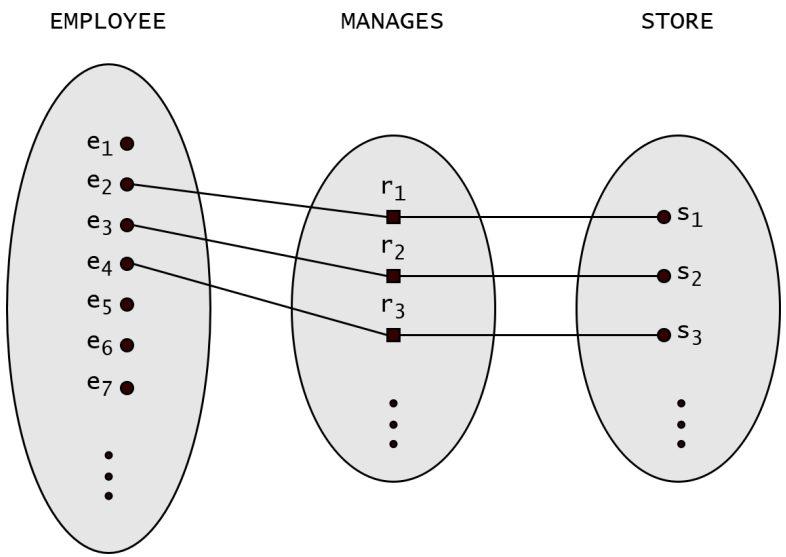
# An example of a relationship

- As an example, the relationship type (or just relationship) `MANAGES` is about "an employee manages a store".
- The relationship instance $r_1$ means employee $e_2$ manages store $s_1$.



# Participation and degree

- For each relationship type, some entity types **participate** (參與) in it.
  - The number of participating entity types is called the **degree** (連結數) of the relationship type.
- Consider the relationship `MANAGES` again.
  - `MANAGES` is of degree two.
  - The relationship instance $r_1$ is participated by entity instances $e_2$ and $s_1$.
- A degree-2 relationship can also be called as a **binary** relationship

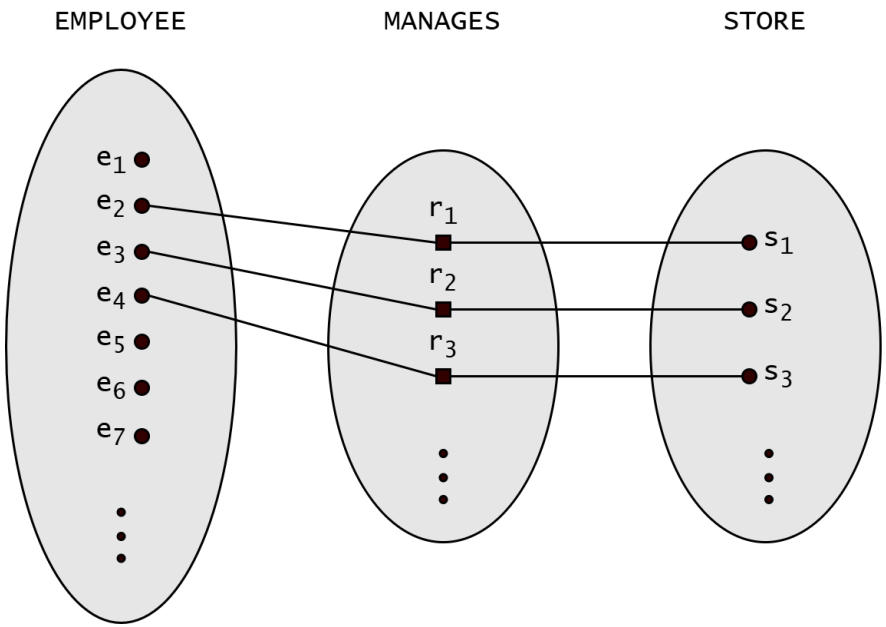


# Cardinality ratios

- Each relationship type has its **cardinality ratio** (參與數比例).
- For degree-2 relationships, possible cardinality ratios are:
  - **1:1 (one to one)**: Each entity instance participates in exactly one relationship instance.
  - **1:*N* (one to many)**: entity instances at one side may participate in any number of instances.
  - ***M*:*N* (many to many)**: entity instances at both sides may participate in any number of instances

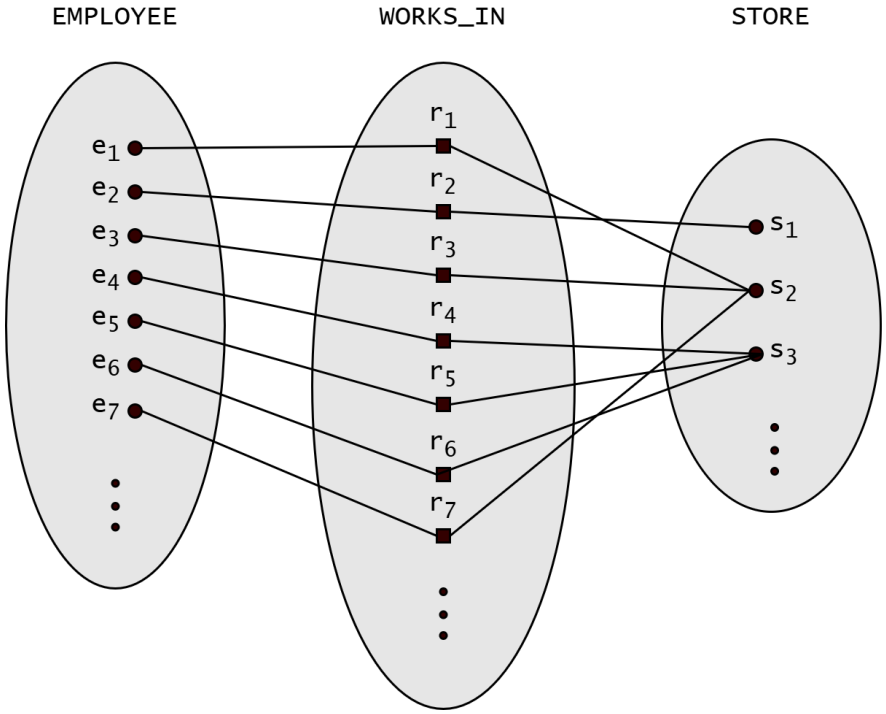
# The 1:1 relationship MANAGES

- Recall the degree-2 relationship MANAGES.
- Suppose that:
  - An employee to manage at most one store, and a store cannot have two managers.
  - MANAGES is then a **1:1** relationship.
- What if an employee can manage more than one store?
- Note that it is possible for some entity instances not to participate in a relationship instance of a type.



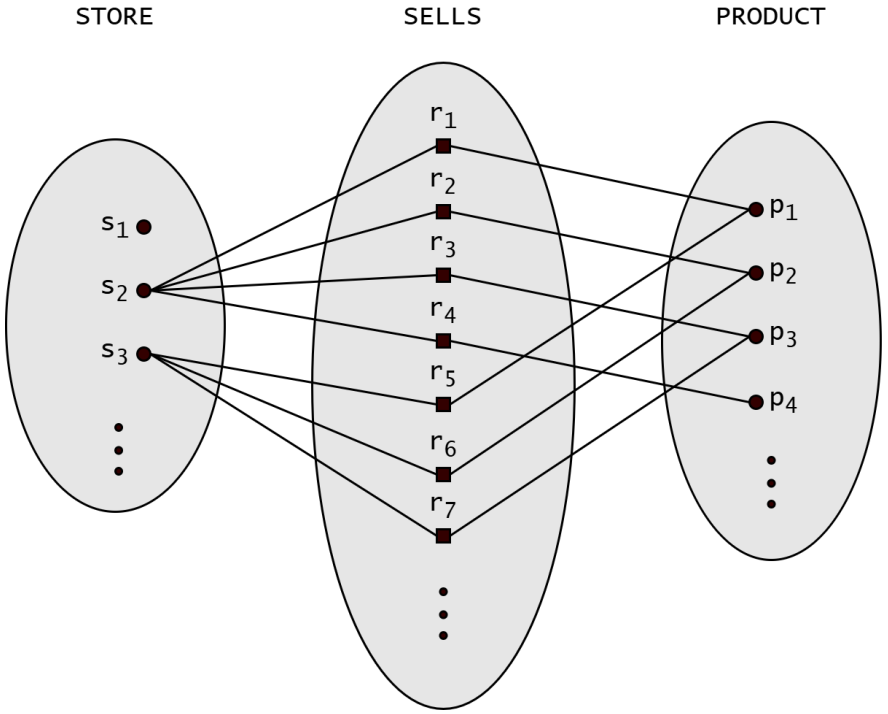
# The 1:*N* relationship WORKS_IN

- The relationship WORKS_IN says "an employee works in a store".
- WORKS_IN is of degree two.
- WORKS_IN is a **1:*N*** relationship:
  - An employee cannot belong to more than one store.
  - A store can have multiple employees.



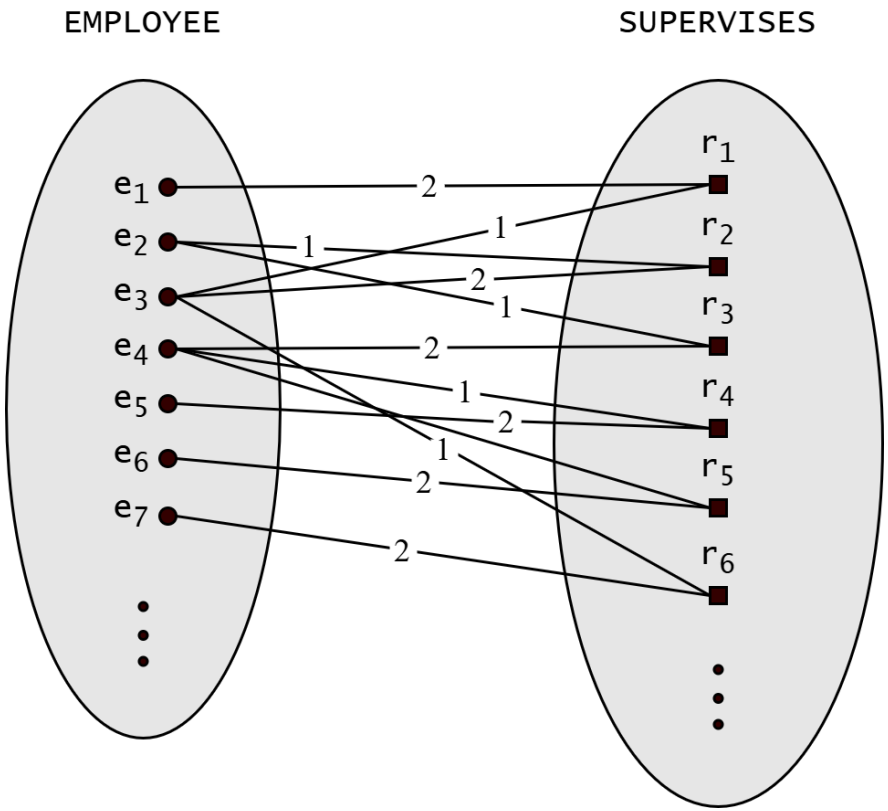
# The *M*:*N* relationship SELLS

- The relationship SELLS says "a store sells several products".
- SELLS is of degree two.
- SELLS is a ***M*:*N*** relationship:
  - A store can sell multiple products.
  - A product can be sold in many stores.



# The recursive relationship SUPERVISES

- A relationship may be **recursive**, i.e., self-referencing.
- The degree-2 relationship SUPERVISES says "an employee supervises another employee".
- The EMPLOYEE entity plays two roles, **supervisor** and **supervisee**, at the same time.
  - 1: supervisor.
  - 2: supervisee.
- What is the cardinality ratio of SUPERVISES?



# Participation constraints

- **Participation constraints** (參與限制) specify whether an entity instance may exist **without** getting connected to another entity instance by participating in a relationship.
- EMPLOYEE has a **total participation** (全體參與) in WORKS_IN .
  - Business rule: All employees must work.
  - Total participation: Each employee instance must participate in at least one WORKS_IN relationship instance.
- EMPLOYEE has a **partial participation** (部份參與) in MANAGES.
  - Business rule: Only part of the employees manages stores.
  - Partial participation: An employee instances can participate in zero MANAGES relationship instance.
  - Note that the participation of STORE in MANAGES is total.

# Identifying relationships

- Recall that we have strong and weak entity types.
  - A weak entity type does not have its own key attributes.
  - It must be connected to an **identifying (owner) entity type** (識別實體類別) through an **identifying relationship** (識別關聯).
- In our example:
  - There must be an identifying relationship between `SALES_DETAIL` to `SALES` and another one between `SALES_DETAIL` and `PRODUCT`.
  - A sales transaction and a product together **identify** a sales detail.
- A weak entity type always **totally participates** in its identifying relationships.
- A weak entity type may also have a **partial key**.
  - A partial key itself cannot uniquely define an entity instance.
  - However, it together with the keys in the identifying entity types can.

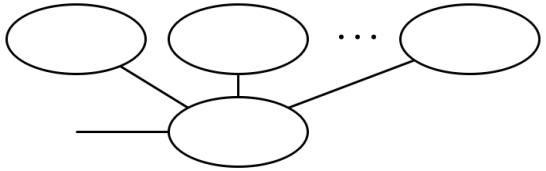
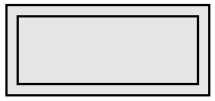
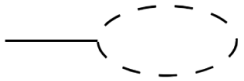
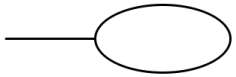
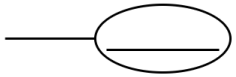
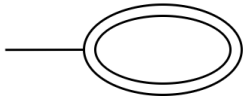
# Road Map

- A motivating example
- Entity-relationship modeling
  - Entities
  - Relationships
  - **Entity-relationship diagram**
- Creating a relational database schema

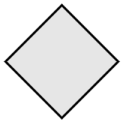
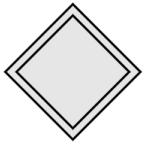
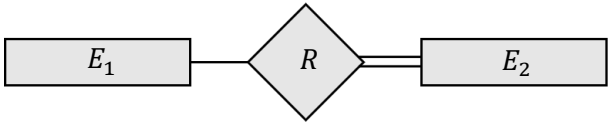
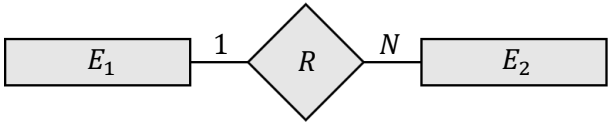
# Entity-relationship diagram

- **Entity-relationship modeling** is the process of specifying entities and relationships.
  - The result is an **entity-relationship model** (**ER model**, 實體關聯模型).
- All of these may be visualized by an **entity-relationship diagram** (**ER diagram**, 實體關聯圖).
- While there is no single standard that everyone must follow, there are commonly-used symbols and naming rules that are suggested.

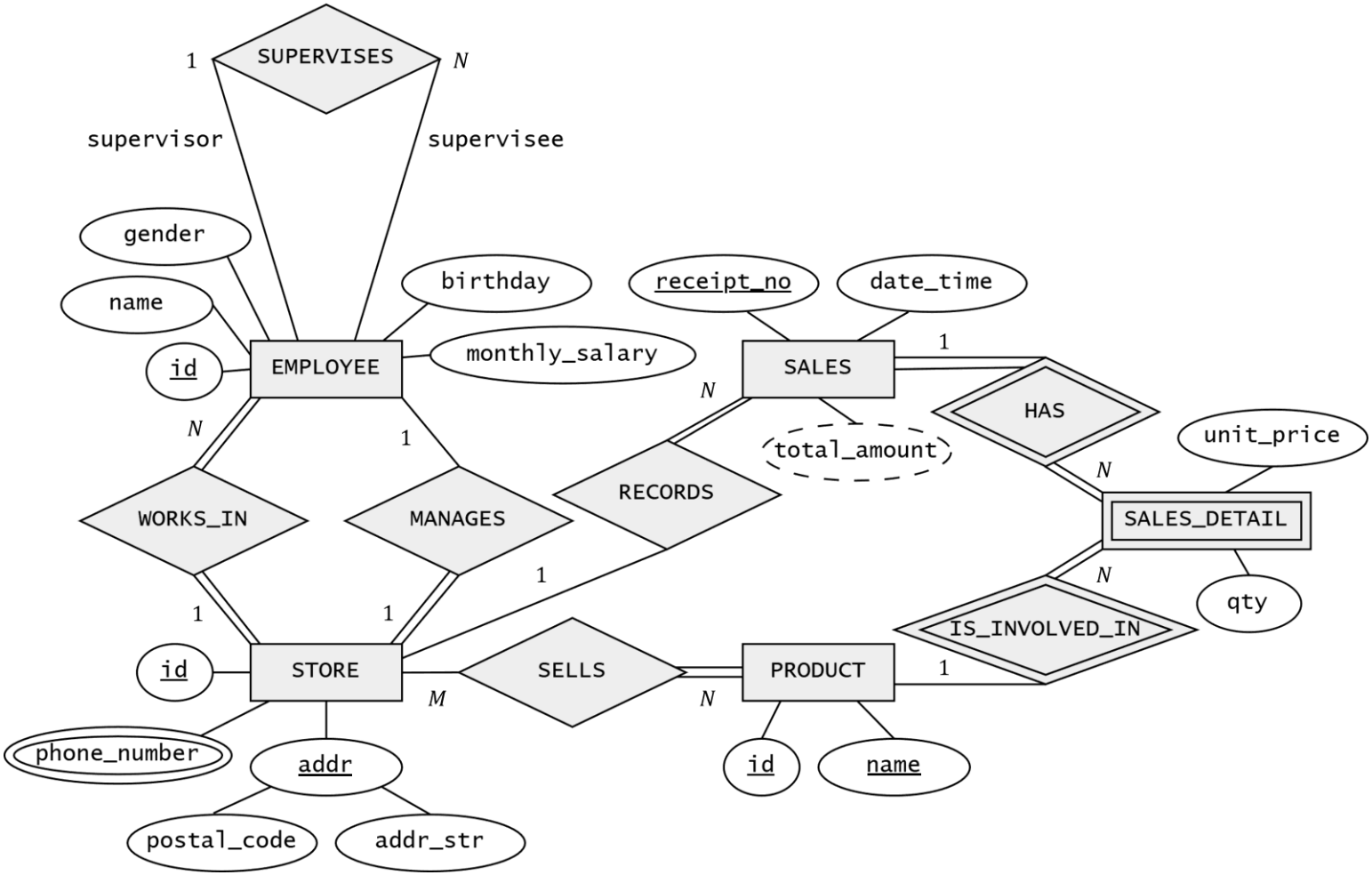
# Suggested symbols in an ER diagram

| Symbol | Meaning | Symbol | Meaning |
|---|---|---|---|
|  | Entity |  | Composite Attribute |
|  | Weak Entity |  | Derived Attribute |
|  | Attribute | | |
|  | Key Attribute | | |
|  | Multivalued Attribute | | |

# Suggested symbols in an ER diagram

| Symbol | Meaning |
| --- | --- |
|  | Relationship |
|  | Identifying Relationship |
|   | Total Participation of $E_2$ in $R$ |
|   | Cardinality Ratio 1:$N$ of $E_1$:$E_2$ in $R$ |

# ER diagram for RETAIL: version 1

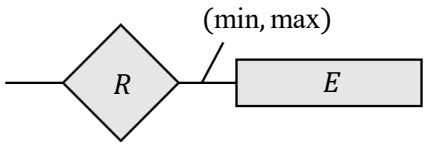


# Some suggestions on naming

- Use **nouns** to be entity type names, attribute names, and use **verbs** to be relationship type names.
- Another naming consideration involves choosing binary relationship names to make the ER diagram readable **from left to right** and **from top to bottom**.
- Some people use only capital characters for entity and relationship names.
  - It is the designer's discretion. It is not required.
  - Nevertheless, one should be **consistent** throughout her/his design.

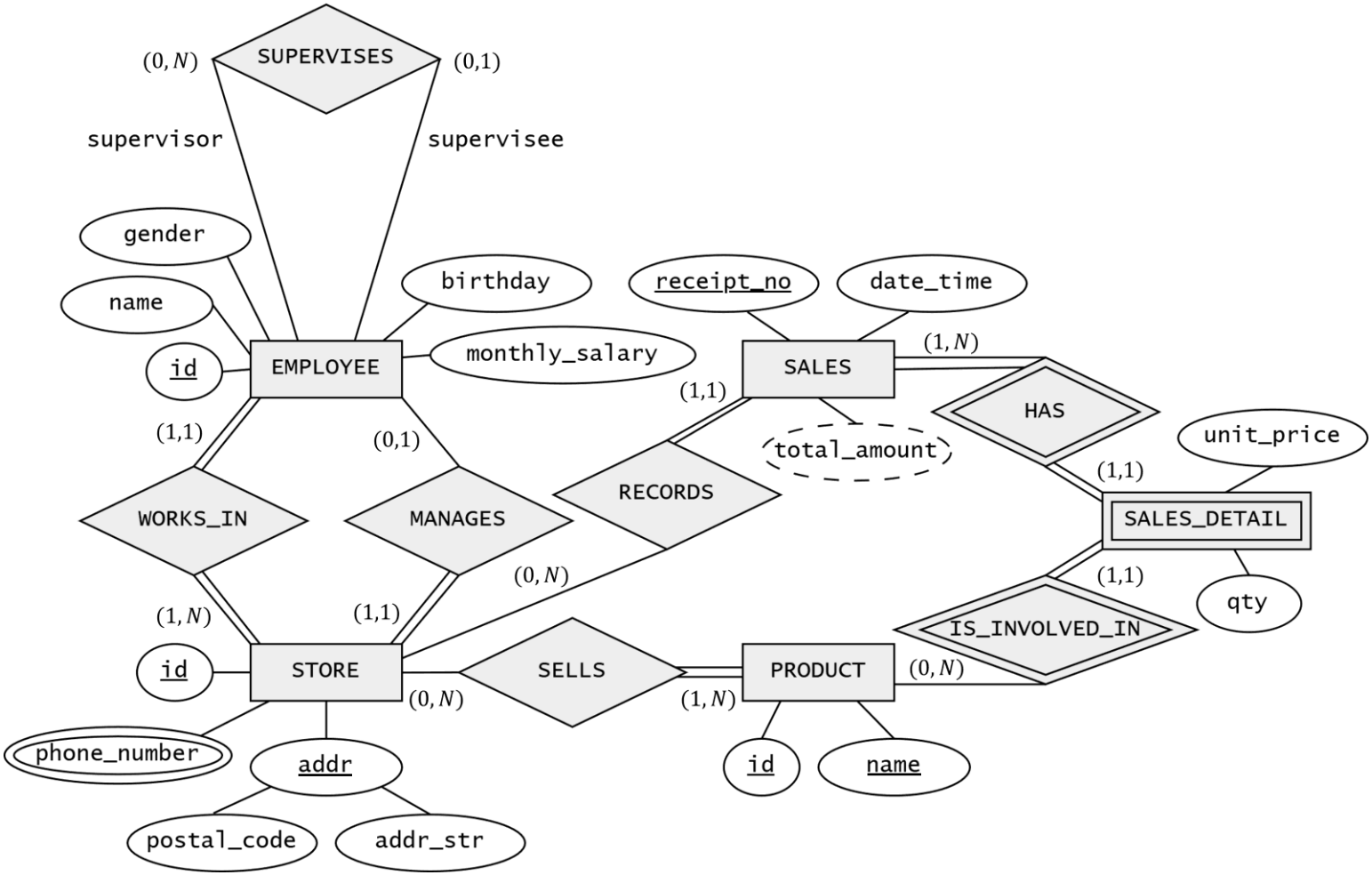
# Another popular format

- Instead of labeling 1, $N$, and $M$ on relationships, some people suggest to use **(min, max)** to provide more information.




Structural Constraint (min, max) on Participation of $E$ in $R$

# ER diagram for RETAIL: version 2

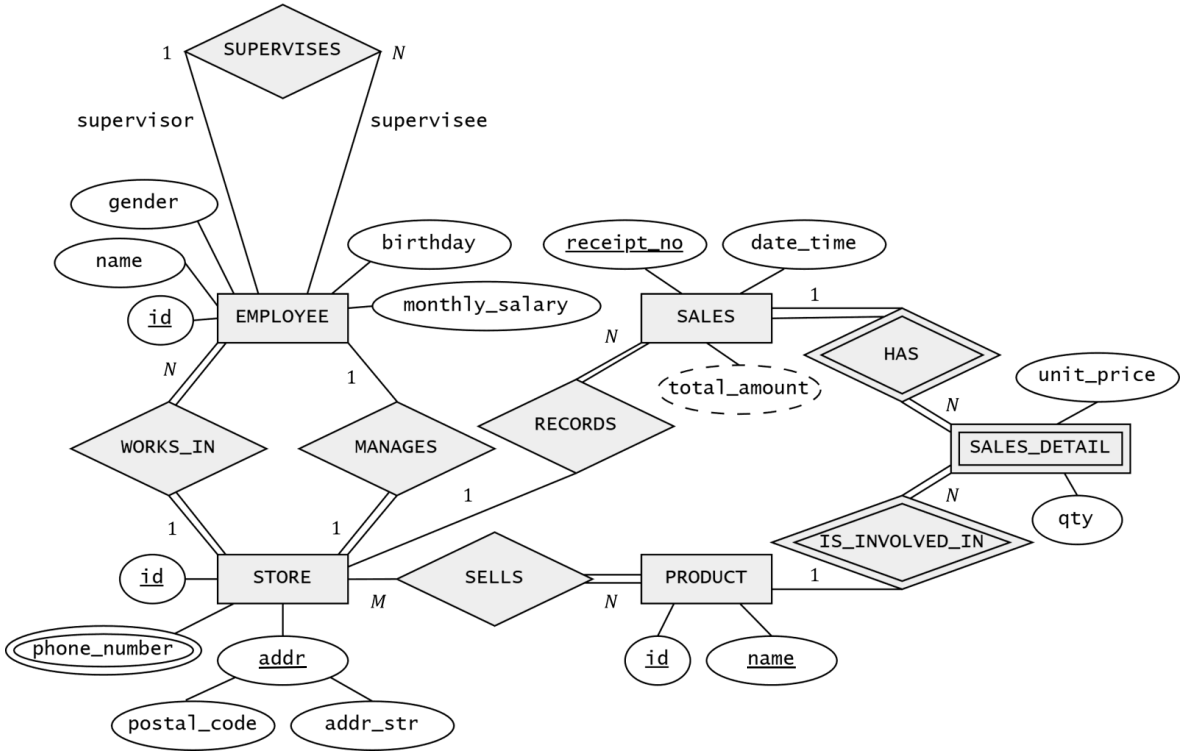


# Road Map

- A motivating example
- Entity-relationship modeling
- **Creating a relational database schema**
  - **ER model, relational model, and relational schema**
  - The ER-relational mapping
  - Summaries and remarks

# From an ER model to a database

- Having an ER model and an ER diagram is great.
- However, an ER model is somewhat **abstract** and **conceptual**.
  - How may we create a **concrete database schema design**?



# Relational model

- While an ER model is abstract and used to represent business rules, it can be converted to a concrete **relational model** (關聯式模型).
- In a relational model, there is a **set of relations** (關係).
- Each entity type in an ER model is represented by a relation.
  - Each relation contains a **set of tuples** (元組), where each tuple represents an **entity instance**.
  - Each **attribute** in a relation represents an **attribute** of an entity type.
- Each relationship type in an ER model is represented by either a relation or some attributes in a relation.

# Relational database

- Every relational model can be implemented by exactly one **relational database schema** (關聯式資料庫綱要).
  - The mapping is one-to-one.
- Each relation is represented by a concrete **data table** (資料表).
  - Each tuple is represented by a **data row** (資料列, **records**) in a table.
  - Each attribute is represented by a **data column** (資料欄, 資料行) in a table.
- A relational database schema is said to include a set of data tables, each having a set of data columns.
  - Designing a relational database schema is identical to constructing a relational data model.

# A short summary

- The concepts and terms are related:

| ER model | Relational model | Relational database |
|---|---|---|
| Entity or relationship type | Relation | Table |
| Entity or relationship instance | Tuple | Row or record |
| Attribute | Attribute | Column |

- A “relationship” is **not** a “relation”!
  - A “relationship” indicates how entities interact with each other.
  - A “relation” is used to describe both entities and relationships.

# Road Map

- A motivating example
- Entity-relationship modeling
- **Creating a relational database schema**
  - ER model, relational model, and relational schema
  - **The ER-relational mapping**
  - Summaries and remarks

# From an ER model to a relational one

- Given an ER model, there is a **step-by-step systematic way** to generate a relational model (and thus a database schema).
  - This is the **ER-relational mapping algorithm**.
- Below we will demonstrate the mapping process using the `RETAIL` example.
  - Because the term "attribute" is used both in an ER model and a relational model, we will use "column" to represent "attribute" in a relational model when it makes the meaning clearer.

# Step 1: Mapping of regular entity types

- For each **regular entity type** *E* in the ER model, create a **relation** *R*.
  - Includes all the simple **attributes** of *E* as **columns** of *R*.
  - Include only the simple component attributes of a composite attribute.
- Choose one (set) of the **key attributes** of *E* as the **primary key** (主鍵) for *R*.
  - A relation has **only one** primary key (where an entity type may have multiple keys), which may be formed by **one or multiple columns**.
  - If the chosen key of *E* is composite, the set of simple attributes that form it will together form the primary key of *R*.
  - If a key attribute is not chosen as the primary key, eventually a **uniqueness constraint** will be set in the DBMS on it to ensure that all entity instances have different values in this key attribute.
- Anyway, primary keys in relational models and keys in ER models are related but not the same thing!

# Example of step 1

- For example, we create the relations `EMPLOYEE`, `STORE`, `PRODUCT`, and `SALES` for the regular entity types `EMPLOYEE`, `STORE`, `PRODUCT`, and `SALES` in the ER model `RETAIL`.
- Then, we choose employee's `id`, store's `id`, product's `id`, and sales' `receipt_no` as primary keys for the relations `EMPLOYEE`, `STORE`, `PRODUCT`, and `SALES` respectively.

**EMPLOYEE**

| id | name | gender | birthday | monthly_salary |
|---|---|---|---|---|

**STORE**

| id | postal_code | addr_str |
|---|---|---|

**PRODUCT**

| id | name |
|---|---|

**SALES**

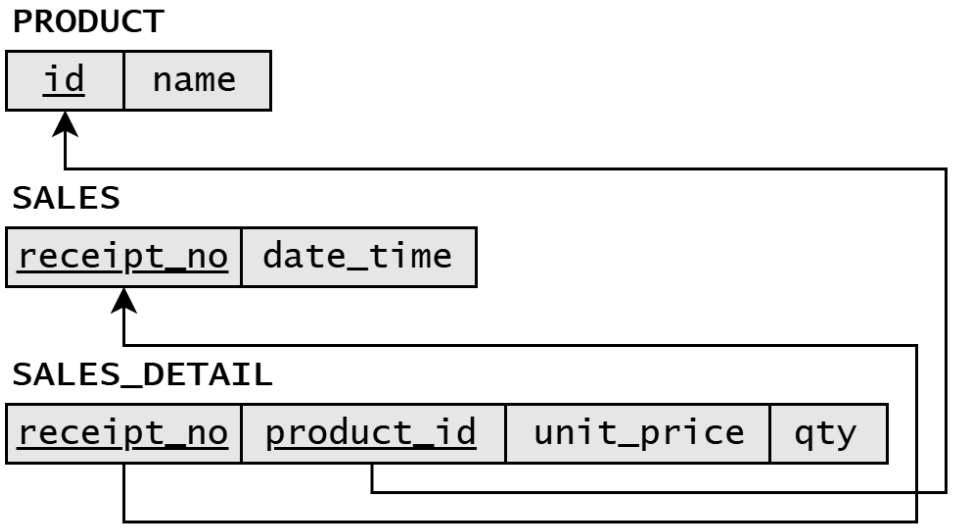
| receipt_no | date_time |
|---|---|

# Step 2: Mapping of weak entity types

- For each **weak entity type** $W$ in the ER model, create a **relation** $R$.
  - Include all simple attributes of $W$ as columns of $R$.
- For each identifying (owner) entity type $E$ of $W$, add column(s) into $R$ by copying the primary key attribute(s) of the relation that correspond to $E$.
  - Set these attributes as **foreign keys** (外部鍵) of $R$ that reference to the primary keys of those identifying entity type(s).
  - In general, for a foreign key $A \rightarrow B$, a value can exist in column $A$ only if it exists in column $B$.
- Set the **primary key** of $R$ as the combination of:
  - The primary key(s) of the identifying entity type(s).
  - The partial key (if any) of the weak entity type $W$.

# Example of step 2: SALES_DETAIL

- For example, we create the **relation** SALES_DETAIL in this step.
- Include the primary key receipt_no of the SALES and the primary key id of the PRODUCT as a **foreign key** attribute of SALES_DETAIL.
  - Rename the column id in PRODUCT as product_id in SALES_DETAIL to make its meaning clear.



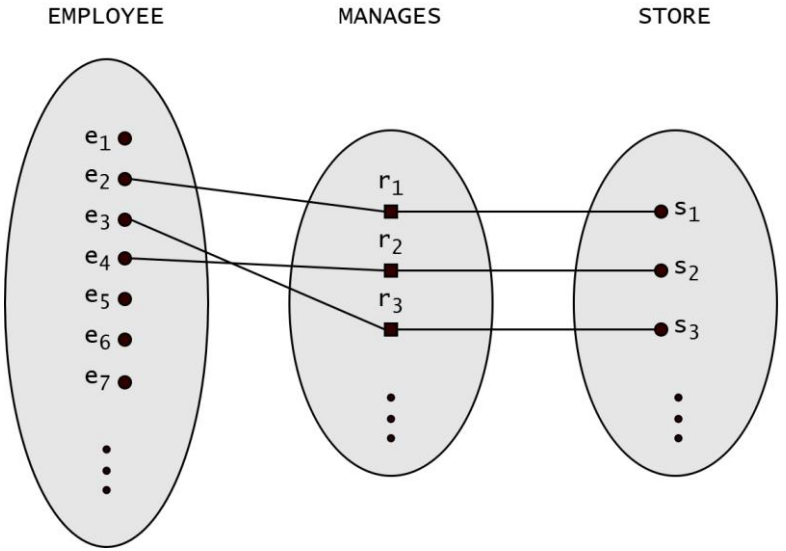

- The **primary key** of SALES_DETAIL is the combination {receipt_no, product_id} so that each entry is unique.

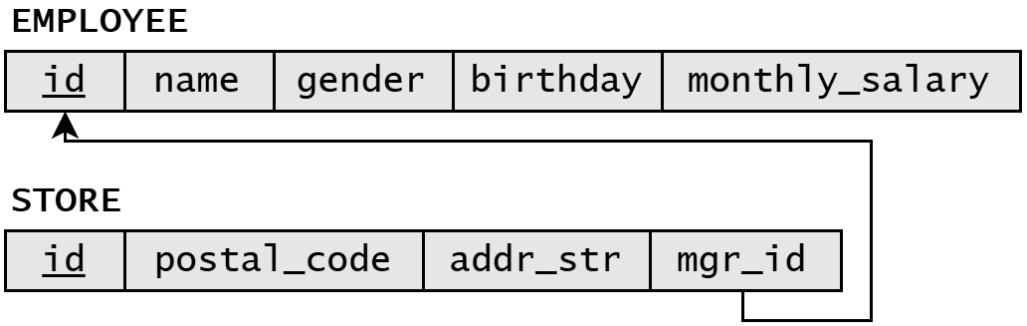
# Step 3: 1:1 relationship types

- For each **1:1** relationship type $R$ in the ER model:
  - Identify the relations $S$ and $T$ that correspond to the entity types participating in $R$.
- Consider the participation constraints:
  - If the participations of $S$ and $T$ are **partial** and **total**, respectively, include the primary key of relation $S$ as a **foreign key** in relation $T$.
  - If the participations of $S$ and $T$ are both partial, do the same thing on either $S$ or $T$ and allow null values in the newly added column(s).
  - If the participations of $S$ and $T$ are both total, merge the two entity types and the relationship into a single relation.
- Include all the simple attributes (if any) of $R$ as columns of $S$.

# Example of step 3: MANAGES

- Consider the 1:1 relationship type MANAGES.
  - EMPLOYEE's participation is partial.
  - STORE's participation is total.
- Include the primary key id of EMPLOYEE as **foreign key** in the STORE and rename it as mgr_id.



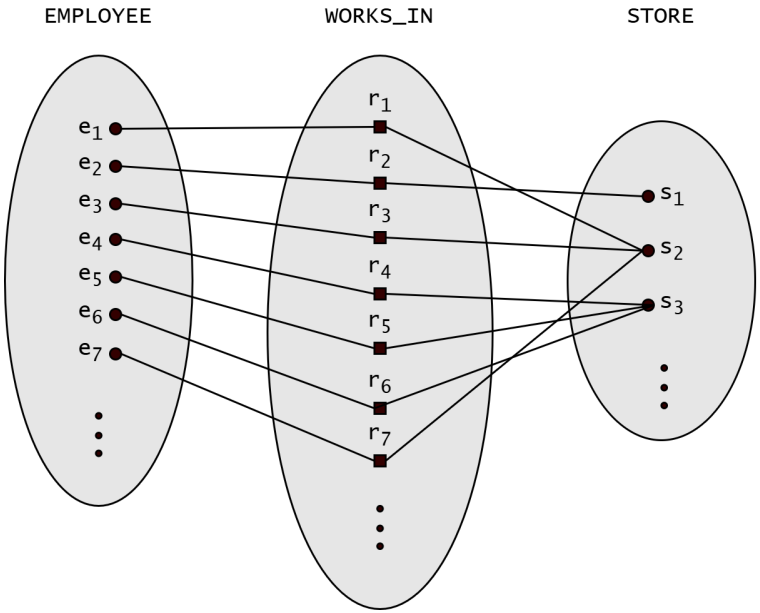



# Step 4: 1:$N$ relationship types

- For each **1:$N$** relationship type $R$:
  - Let $E_1$ and $E_2$ be the two participating entity types where "each $E_2$ instance can be related to at most one $E_1$ instance."
- Include the primary key of $E_1$ as a **foreign key** in $E_2$.
- Include any simple attributes in $R$ as columns of $E_2$.

# Example of step 4: WORKS_IN

- For the 1: *N* relationship WORKS_IN:
  - Include the primary key id of STORE as a **foreign key** in EMPLOYEE and call it store_id.




EMPLOYEE

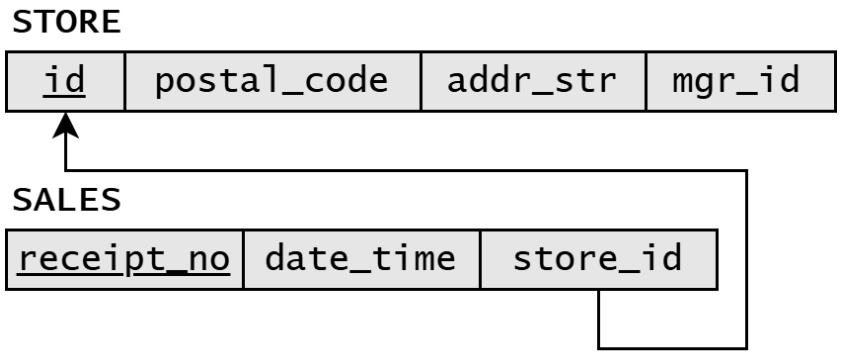
| id | name | gender | birthday | monthly_salary | supervisor_id | store_id |
|---|---|---|---|---|---|---|

STORE

| id | postal_code | addr_str | mgr_id |
|---|---|---|---|

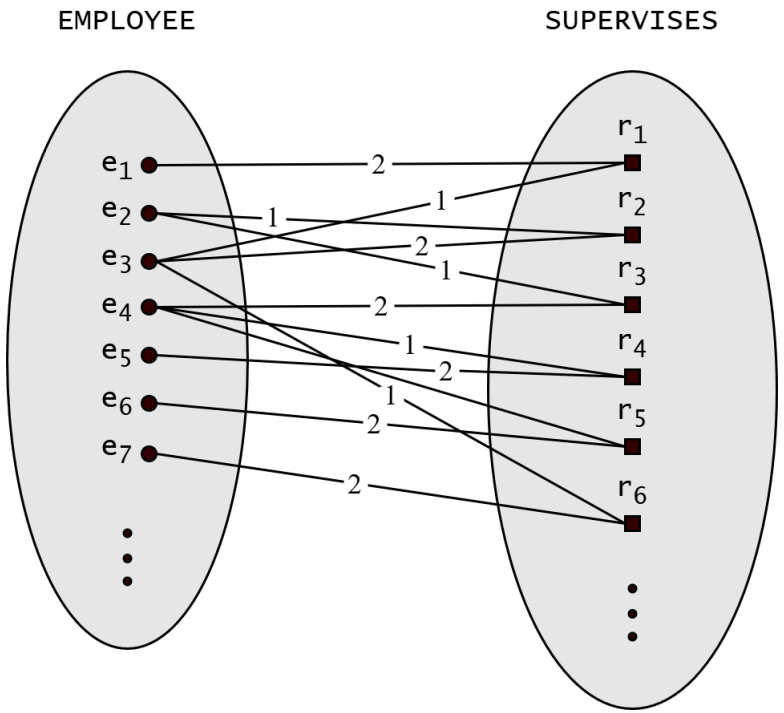
# Example of step 4: RECORDS

- For the 1: *N* relationship RECORDS:
  - Include the primary key id of STORE as a **foreign key** in SALES and call it store_id.



# Example of step 4: SUPERVISES

- For the recursive relationship type SUPERVISES:
  - Include the primary key id of EMPLOYEE as **a foreign key in itself** and call it supervisor_id.




EMPLOYEE

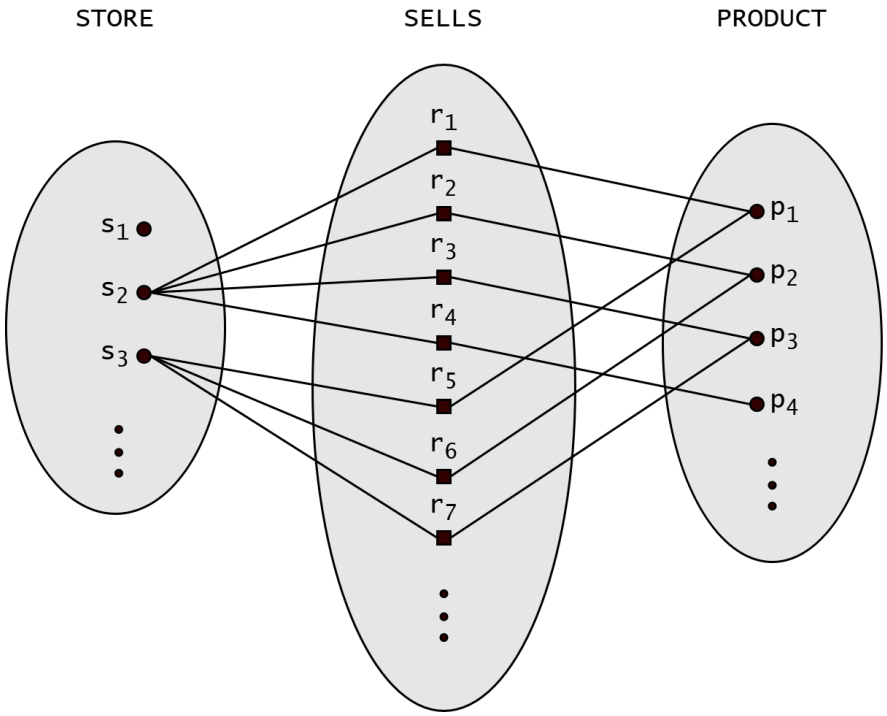
| id | name | gender | birthday | monthly_salary | supervisor_id |
|---|---|---|---|---|---|

# Step 5: $M$:$N$ relationship types

- For each ***M*:*N*** relationship type $R$:
  - Create a new **relation** $S$ to represent $R$.
  - Include the primary keys of the relations that represent the participating entity types as foreign key attributes in $S$; use **their combination** to form the **primary key** of $S$.
  - Note that we cannot represent an $M$:$N$ relationship type by a single foreign key attribute. A new relation is needed.
- Include all simple attributes of the relationship type $R$ as columns of the relation $S$.

# Example of step 5: SELLS

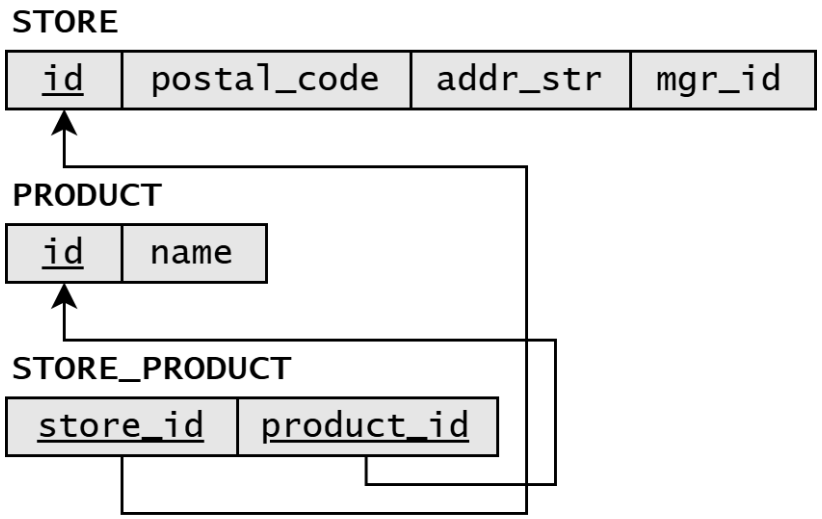
- Consider the $M$:$N$ relationship type SELLS.




- Adding a column in either STORE or PRODUCT is not enough.

# Example of step 5

- We create a **new relation** STORE_PRODUCT.
- We include the **primary keys** of the STORE and PRODUCT relations as **foreign keys** in STORE_PRODUCT and rename them store_id and product_id, respectively.

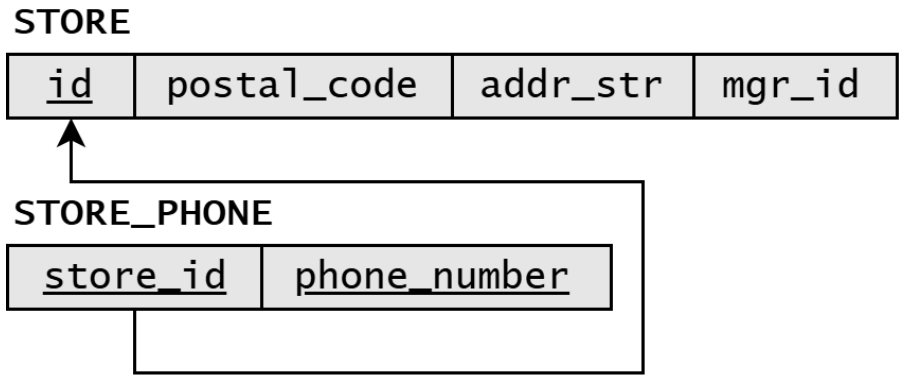


# Step 6: Multi-valued attributes

- For each **multi-valued attribute** $A$ in an entity type $E$, create a new **relation** $R$.
  - Include the primary key $K$ of the relation representing the entity type $E$ as a foreign key in the new relation $R$.
  - Include also a column to represent the attribute $A$.
- The **primary key** of $R$ is the **combination** of $A$ and $K$.

# Example of step 6: phone_number

- For the multi-valued attribute phone_number.
- Create a relation STORE_PHONE.
- The primary key of STORE_PHONE is the combination of {store_id, phone_number}.



# Step 7: $n$-ary relationship types

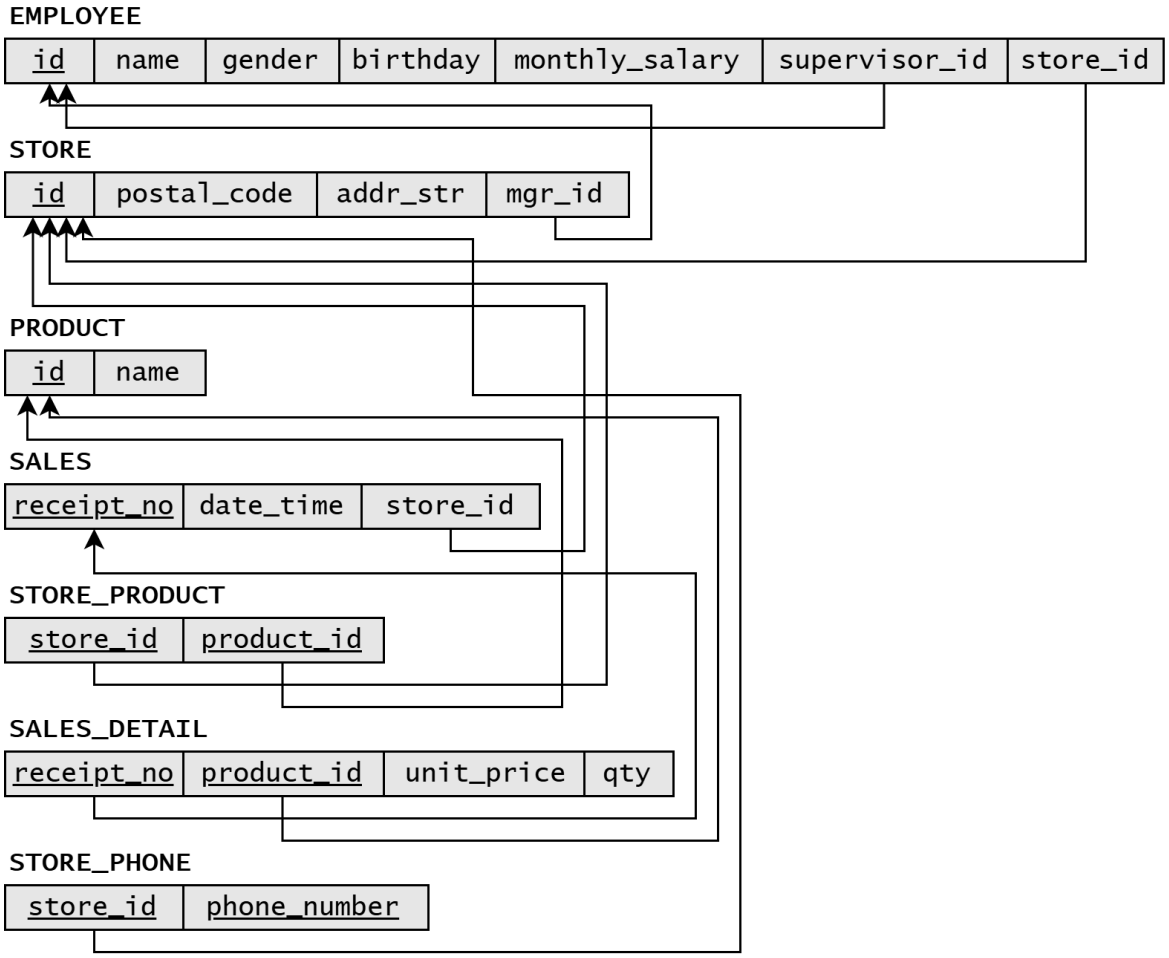
- For each **$n$-ary relationship type** $R$ where $n > 2$, create a new **relation** $S$ to represent $R$.
  - Include the primary keys of the relations that represent the participating entity types as **foreign key** attributes in $S$.
  - Let the **primary key** of $S$ be the combination of all these foreign keys.
- Include any simple attributes of the $n$-ary relationship type as attributes of $S$.

# Example of step 7

- For the `RETAIL` example, we do not need $n$-ary relationships with $n > 3$.
- Let's imagine the following:
  - Suppose that in a university, the relationship `TAKE` indicates "a student takes a course in a semester", where `STUDENT`, `COURSE`, and `SEMESTER` are all entities.
  - The relationship `TAKE` is **ternary** (三元).
  - In the relational model, there will be three relations `STUDENT`, `COURSE`, and `SEMESTER`. There will also be a relation `TAKE` consisting of the **primary keys** of these three relations (e.g., `student_id`, `course_id`, and `semester_id`) and maybe **some other attributes** (e.g., `grade`).

# Relational database schema of RETAIL

- The result is a **relational database schema**, which can be represented by a **relational schema diagram** (關聯式綱要圖), or simply schema diagram.
  - Tables.
  - Columns.
  - Primary keys.
  - Foreign keys.



# Road Map

- A motivating example
- Entity-relationship modeling
- **Creating a relational database schema**
  - ER model, relational model, and relational schema
  - The ER-relational mapping
  - **Summaries and remarks**

# Summaries about data modeling

- **Data modeling** can be considered as a step of **designing** a database.
- The typical design process:
  - **ER modeling**: Collecting and understanding the business rules. Representing the business rules by an ER model. Using **ER diagrams** to visualize an ER model.
  - **Relational data modeling** or **relational schema design**: Converting an ER model to a relational data model (which is equivalent to a relational schema). Using **relational schema diagrams** to visualize a relational schema.
  - **Creation of concrete data tables**.
- We may then insert **data rows** into data tables.

# Remarks about ER modeling

- A relational model is equivalent to a relational schema.
  - When we create concrete data tables, there are some details (such as data types of columns) to be determined.
  - Still, compared to an ER model, a relational data model is more concrete.
- Multiple ER models can map to the same relational data model.
  - An ER model is like the **pseudocode** of algorithm design, and a relational data model is like the final code we want.
  - There can be **multiple** pseudocode corresponding to the same code.
  - There is **no standard way** to write pseudocode.
- Therefore, there is no need (and no way) to make an ER diagram "correct".
  - Still, being able to create or read an ER model is great.

# Remarks about ER modeling

- In practice, when one needs to design a relational data model:
  - If one **cannot** create a relational data model directly from business requirements, she/he may **start from** ER modeling.
  - If one **can** create a good relational schema design directly, she/he may **skip** ER modeling.
  - In many cases, we skip the ER diagram but do not skip ER modeling (e.g., we need to know whether `MANAGE` is $1:1$ or $1:N$ or even $M:N$).
- Nevertheless, there are still situations that a technical person needs to **describe** a database to technical people (such as other developers) or non-technical people (such as users).
  - An ER diagram is useful, and making one to illustrate the major part of a database is a good idea.
  - An ER diagram is not only a **design tool** but also a **communication tool**!